Amazones



Présenté par
Nicolas Caminade, Tanguy Decabrat, Anya Bazizi, Younes Ouaammou, Thomas-Alexandre Moreau

Sommaire

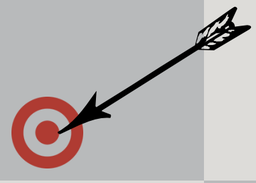


Le jeu

Règles

Histoire

Complexité



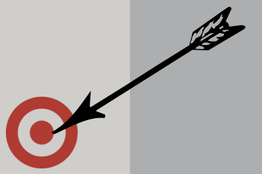
Structures et algorithmes

Logique de jeu

Réseau

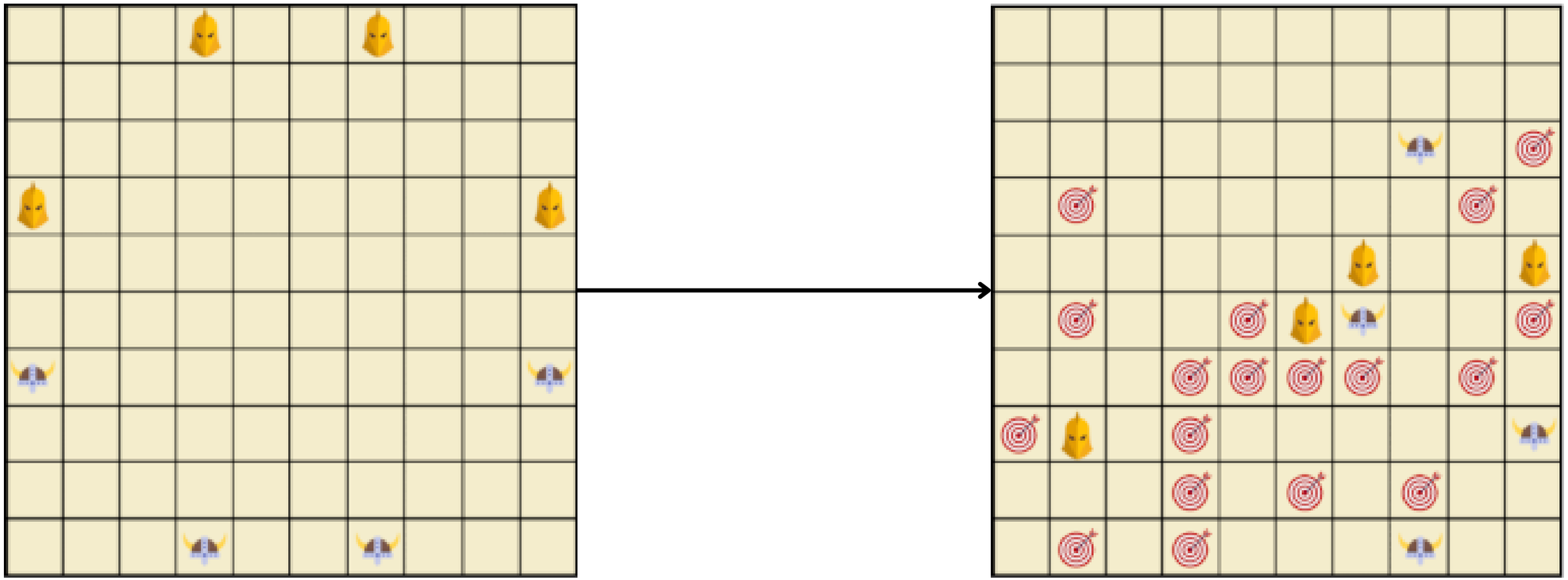
IA

...



Architecture

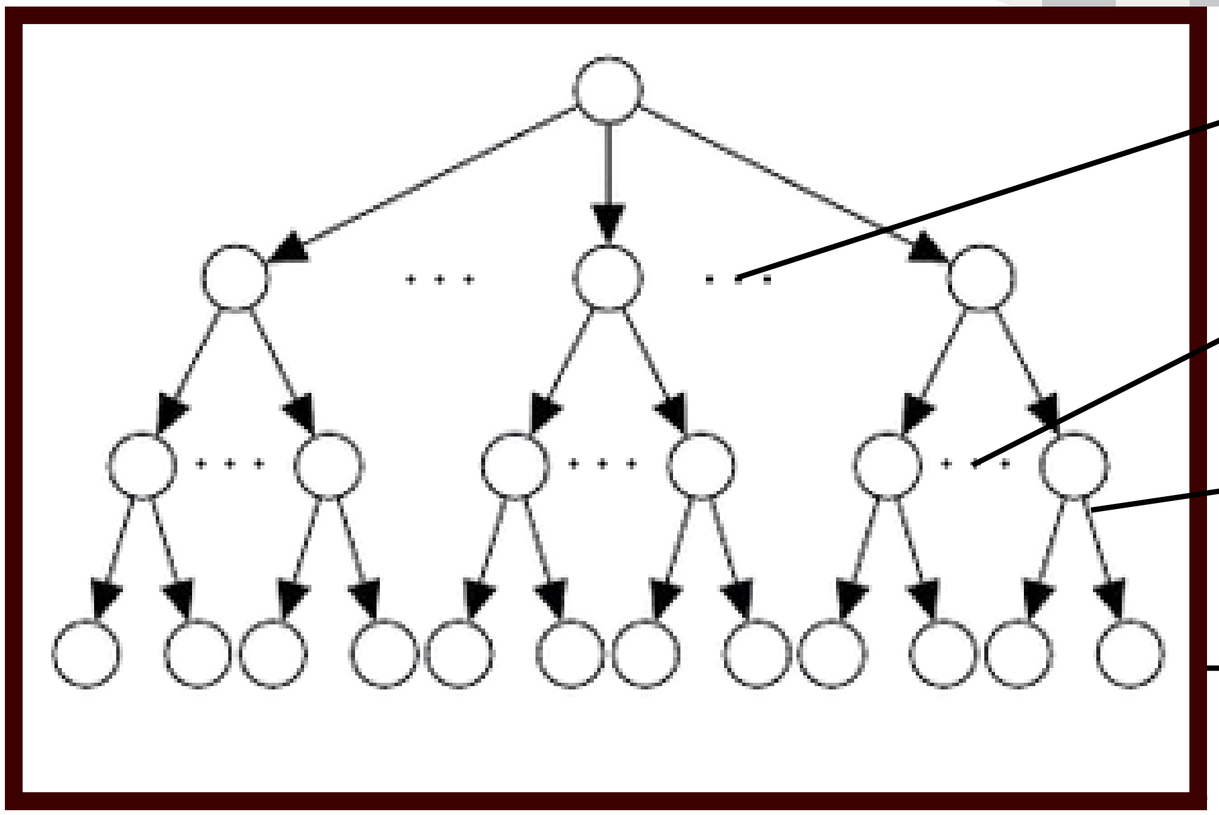
Règles



Histoire du jeu



Complexité



Facteur Branch. Initial : ~2176

Facteur Branch. Moyen: ~374/~299

Nb. Tours Moyen : ~84

Complex. Arbre de jeu : ~10^212

unique + transpose

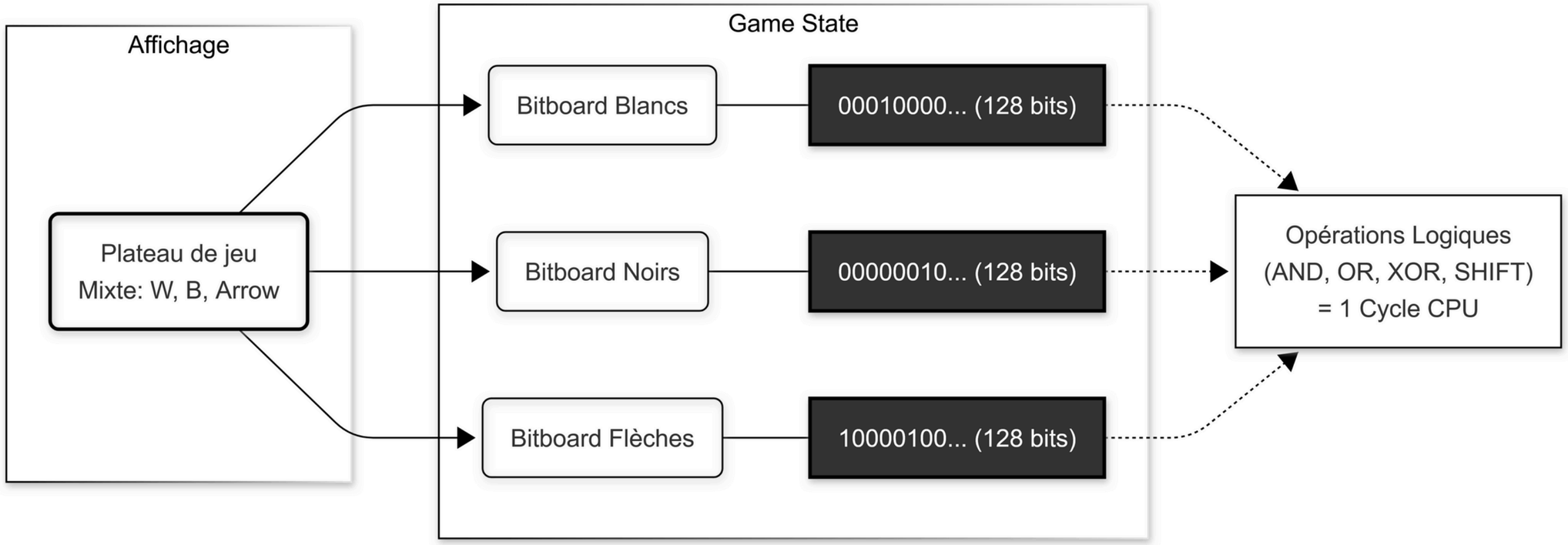
Complex. Espaces d’Etats : ~10^40

Et par rapport à d'autres jeux ?

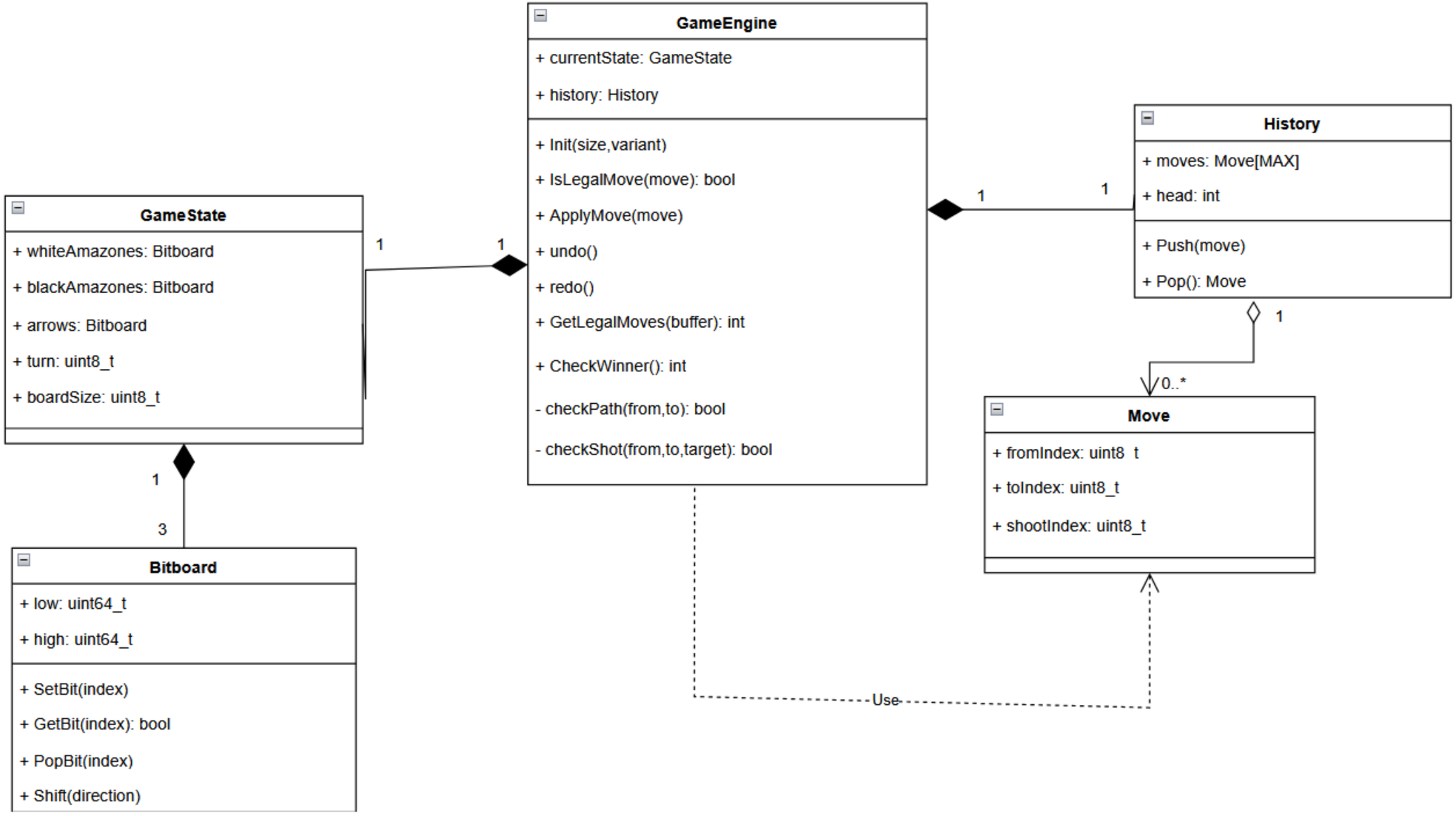
	Échecs	**Amazones**	**Go**
Facteur Branch. Initial	~ 20	~ 2176	~ 361
Facteur Branch. Moyen	~ 35	~ 374 joueur de départ ~ 299 deuxième joueur	~ 250
Nombre de tours moyen	~ 80	~ 84	~ 150
Cmplx. Arbre de jeu	$\sim 10^{123}$	$\sim 1.09 * 10^{212}$	$\sim 10^{360}$
Cmplx. Espaces d'états	de $\sim 3.7 * 10^{43}$ à $\sim 10^{52}$	$\sim 6.45 * 10^{40}$	$\sim 10^{170}$

Algorithmes et Structures de Données

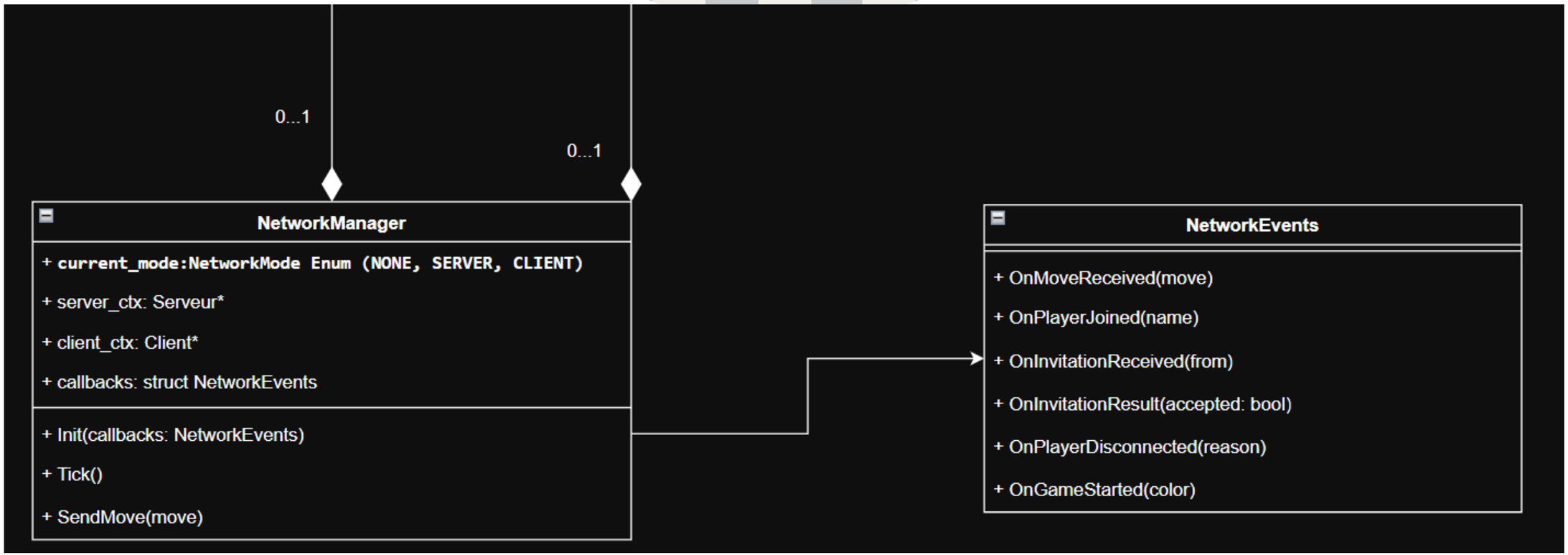
Moteur de jeu



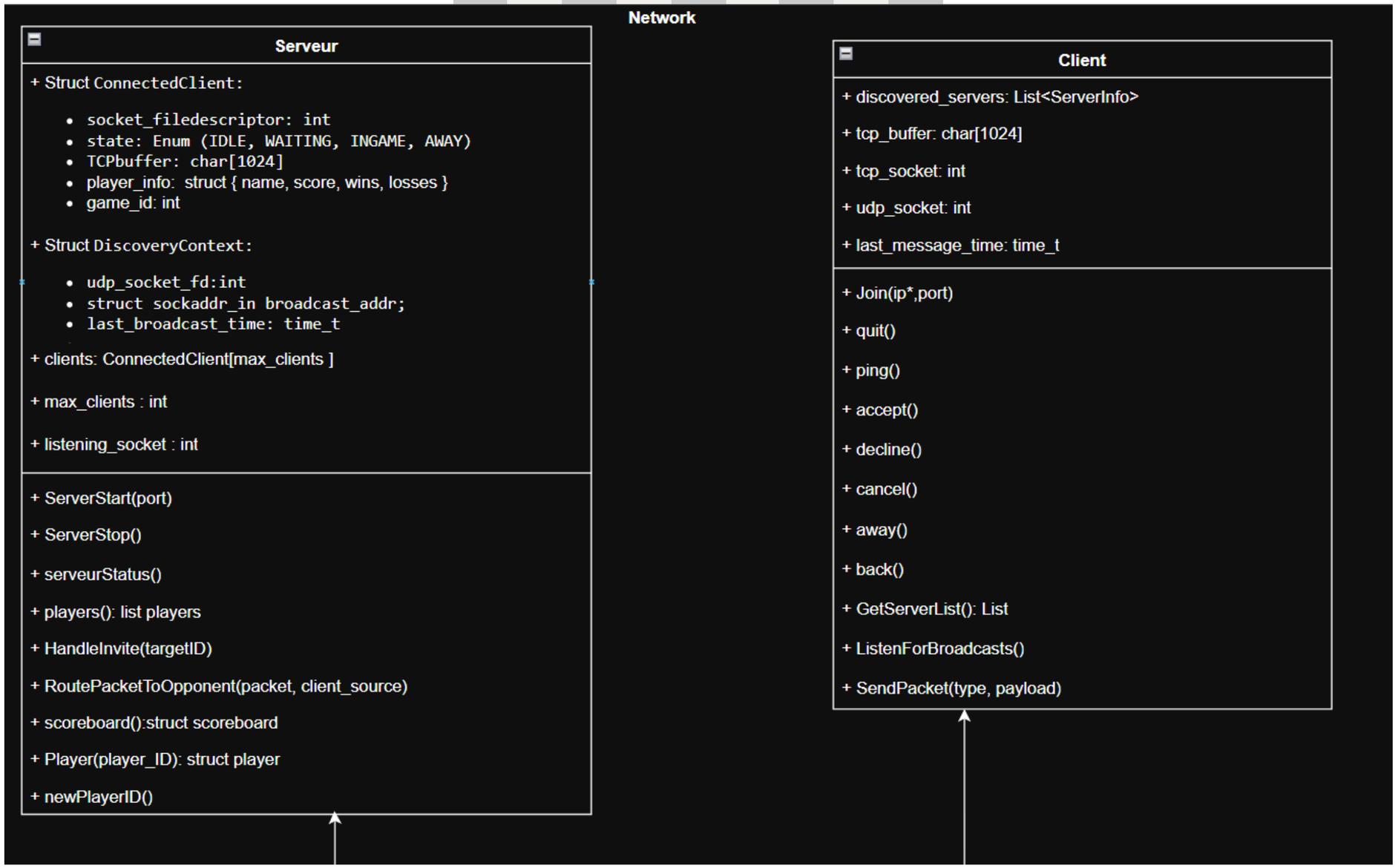
Moteur de jeu



Réseau


0...1
0...1
NetworkManager
+ current_mode:NetworkMode Enum (NONE, SERVER, CLIENT)
+ server_ctx: Serveur*
+ client_ctx: Client*
+ callbacks: struct NetworkEvents
+ Init(callbacks: NetworkEvents)
+ Tick()
+ SendMove(move)
NetworkEvents
+ OnMoveReceived(move)
+ OnPlayerJoined(name)
+ OnInvitationReceived(from)
+ OnInvitationResult(accepted: bool)
+ OnPlayerDisconnected(reason)
+ OnGameStarted(color)

Réseau


Network
Serveur
+ Struct ConnectedClient:
socket_filedescriptor: int
state: Enum (IDLE, WAITING, INGAME, AWAY)
TCPbuffer: char[1024]
player_info: struct { name, score, wins, losses }
game_id: int
+ Struct DiscoveryContext:
udp_socket_fd:int
struct sockaddr_in broadcast_addr;
last_broadcast_time: time_t
+ clients: ConnectedClient[max_clients]
+ max_clients : int
+ listening_socket : int
+ ServerStart(port)
+ ServerStop()
+ serveurStatus()
+ players(): list players
+ HandleInvite(targetID)
+ RoutePacketToOpponent(packet, client_source)
+ scoreboard():struct scoreboard
+ Player(player_ID): struct player
+ newPlayerID()
Client
+ discovered_servers: List<ServerInfo>
+ tcp_buffer: char[1024]
+ tcp_socket: int
+ udp_socket: int
+ last_message_time: time_t
+ Join(ip*,port)
+ quit()
+ ping()
+ accept()
+ decline()
+ cancel()
+ away()
+ back()
+ GetServerList(): List
+ ListenForBroadcasts()
+ SendPacket(type, payload)

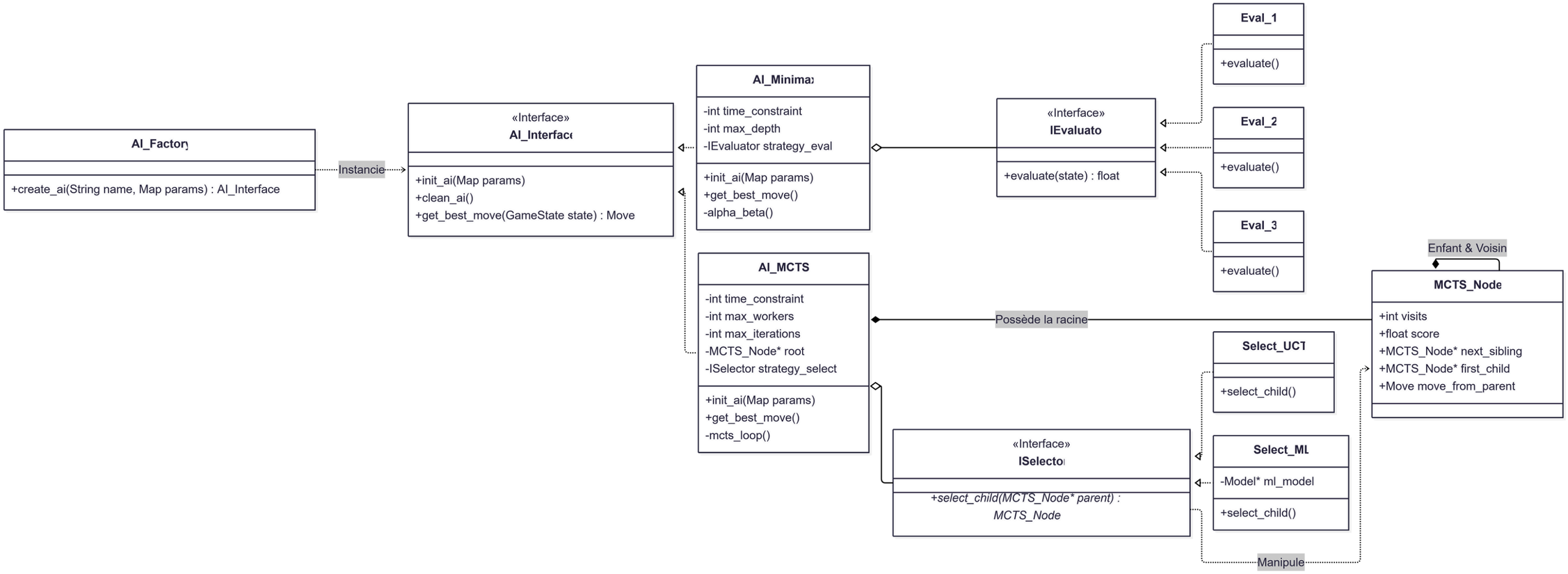
Réseau

Reconstruction TCP

Multiplexage (I/O)

Routage Sélectif

IA


AI_Factory
+create_ai(String name, Map params) : AI_Interface
Instancie
«Interface»
AI_Interface
+init_ai(Map params)
+clean_ai()
+get_best_move(GameState state) : Move
AI_Minimax
-int time_constraint
-int max_depth
-IEvaluator strategy_eval
+init_ai(Map params)
+get_best_move()
-alpha_beta()
«Interface»
IEvaluato
+evaluate(state) : float
Eval_1
+evaluate()
Eval_2
+evaluate()
Eval_3
+evaluate()
AI_MCTS
-int time_constraint
-int max_workers
-int max_iterations
-MCTS_Node* root
-ISelector strategy_select
+init_ai(Map params)
+get_best_move()
-mcts_loop()
Possède la racine
Enfant & Voisin
MCTS_Node
+int visits
+float score
+MCTS_Node* next_sibling
+MCTS_Node* first_child
+Move move_from_parent
«Interface»
ISelecto
+select_child(MCTS_Node* parent) : MCTS_Node
Select_UCT
+select_child()
Select_ML
-Model* ml_model
+select_child()
Manipule

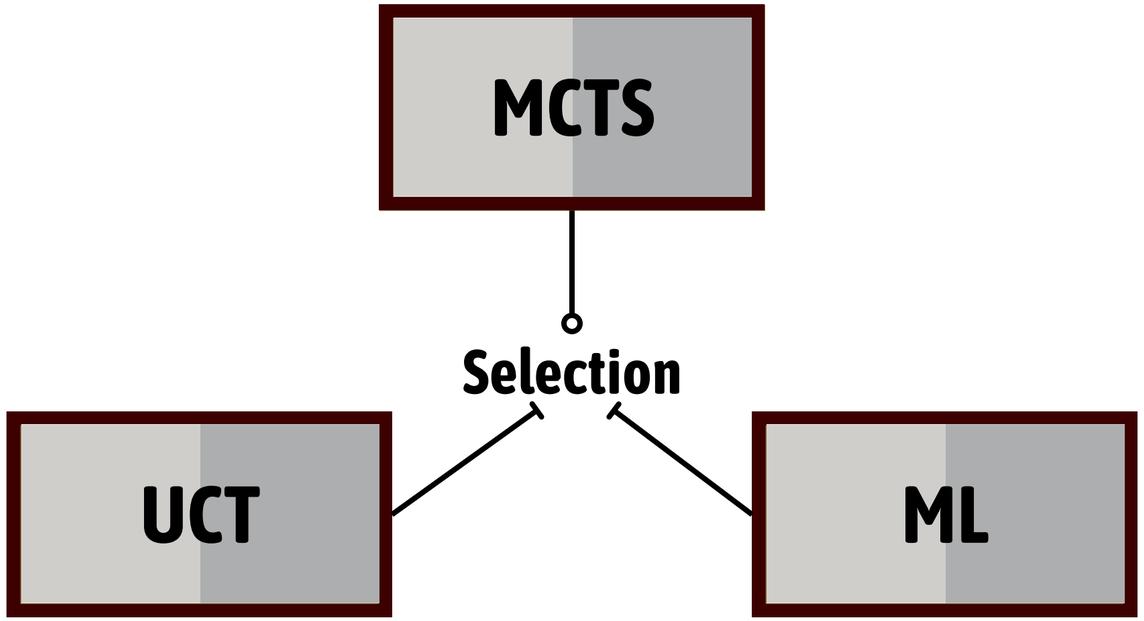
IA

Evaluations

TaM	QD	ME

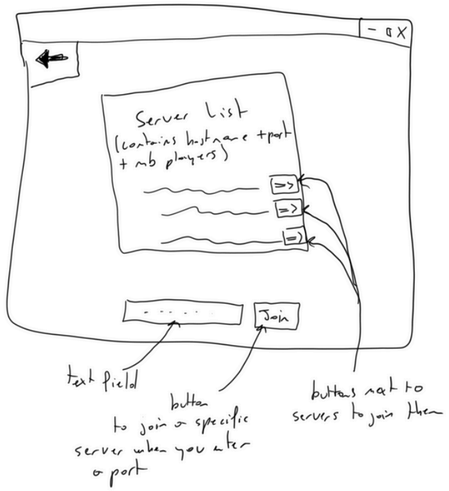
IA

Selections

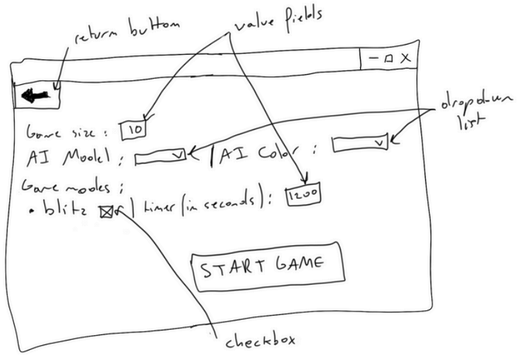


GUI

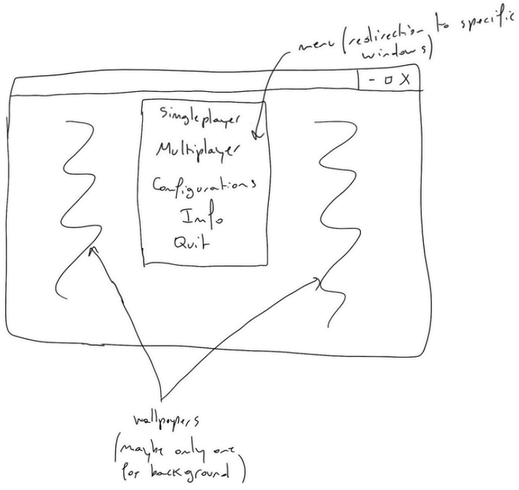
Exemples



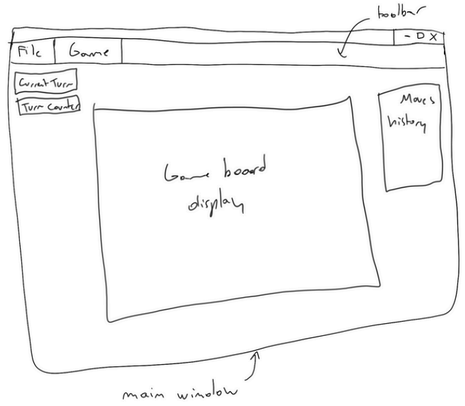
JOIN SERVER
(MULTIPLAYER)



NEW GAME WINDOW
(SINGLEPLAYER)



MAIN MENU

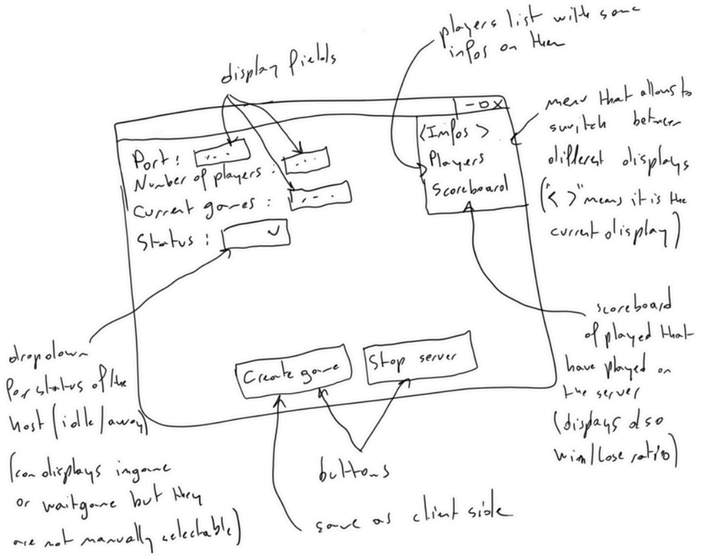


When in a game and press Quit without saving before (in "File" tab or the up-right-cross), it shows a pop-up asking if we want to save before or not

The players get a pop up when a game is finished, telling them if they won or lost

⚠ Undo/Redo disabled in multiplayer, and load/save only available for the "game master"

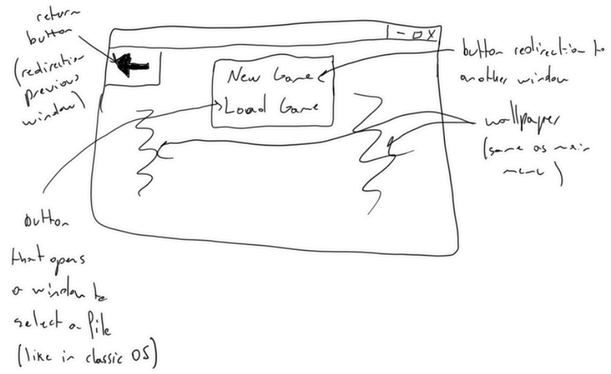
GAME WINDOW



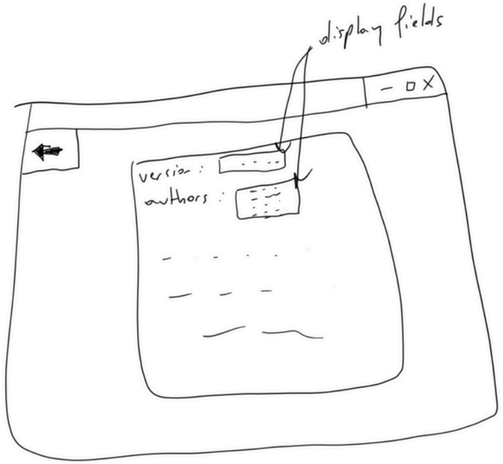
HOST PANEL WINDOW
(MULTIPLAYER)

GUI

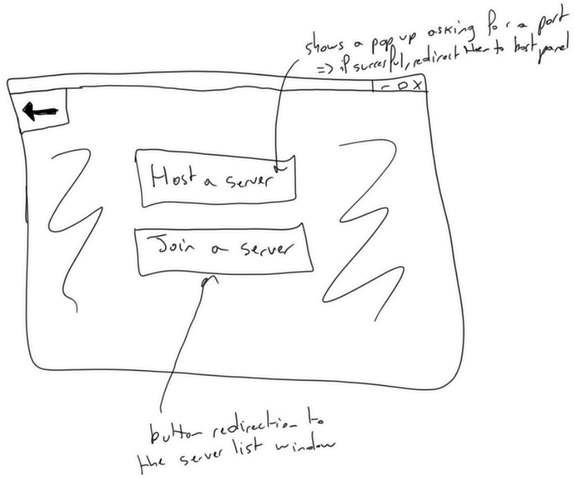
Exemples



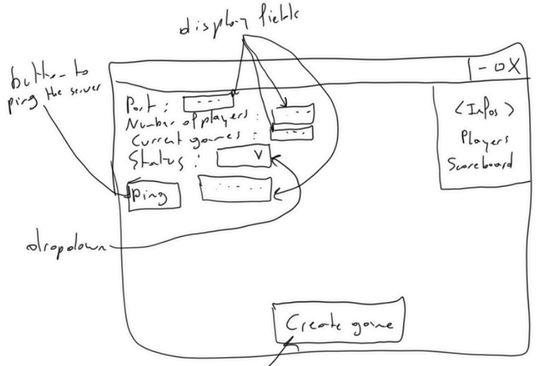
SINGLEPLAYER WINDOW



INFO WINDOW



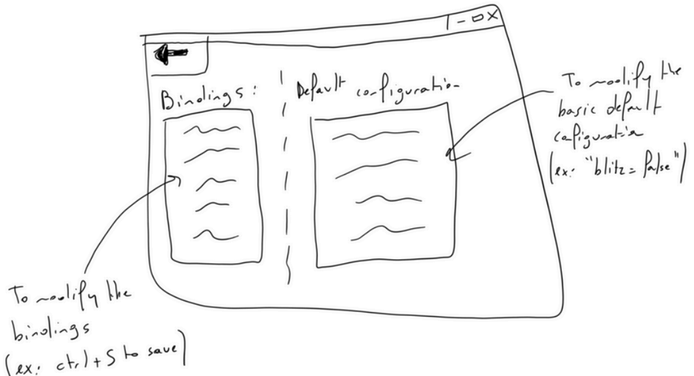
MULTIPLAYER WINDOW



shows a pop up to invite a player, and if the player accepts, the client can start a game.
(the client can also cancel the invitation that they sent)

(but you can also just decide to use one or two AI instead) (and you can set some settings like blitz etc)
the client gets a pop up when they receive or pop to accept or refuse the invitation

CLIENT PANEL WINDOW (MULTIPLAYER)



CONFIGURATIONS PANEL



Gtk 4.x

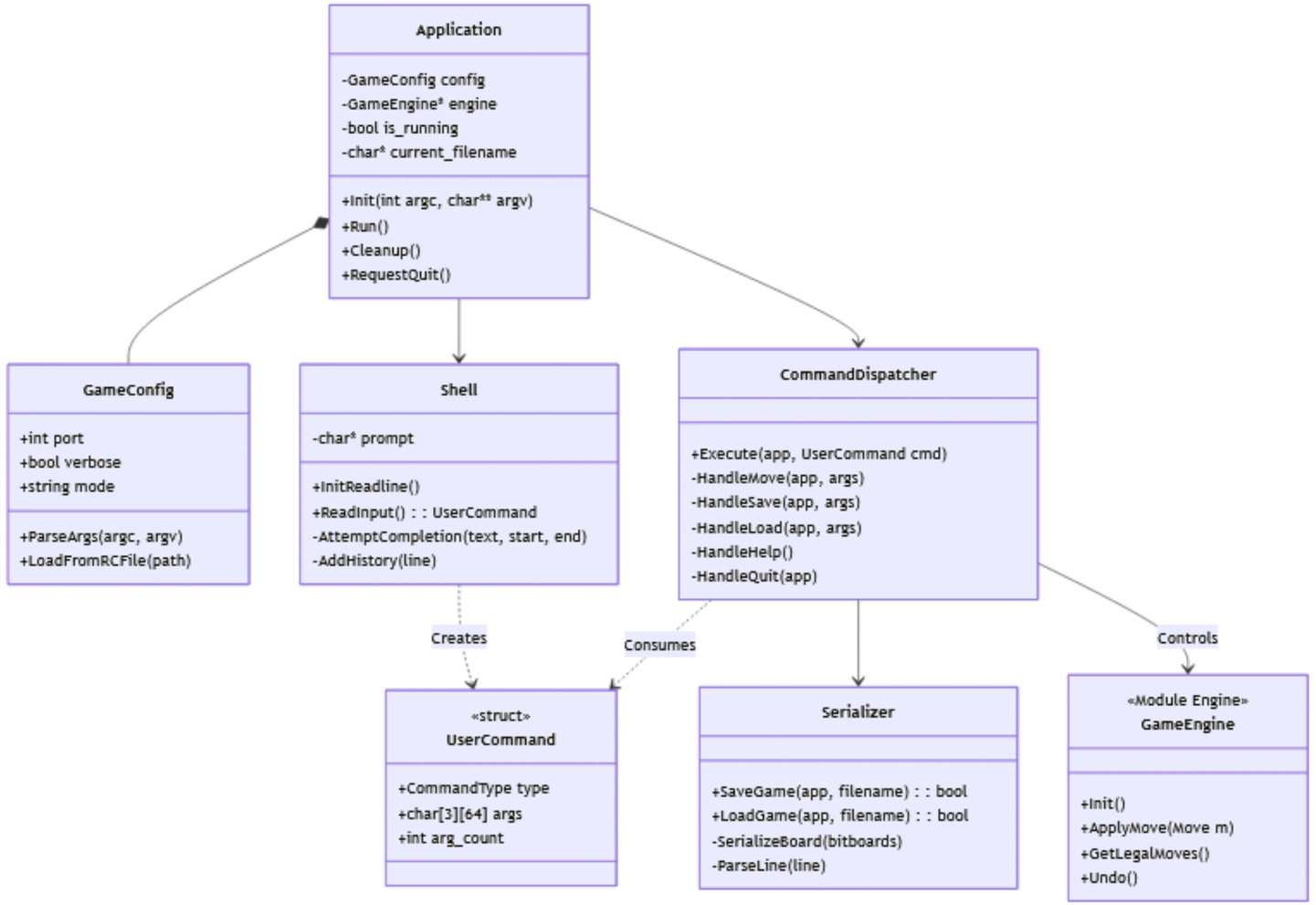
Concept de base

- GtkApplication
- GtkButton
- GtkWidget
- GtkStack

Drag’n’Drop

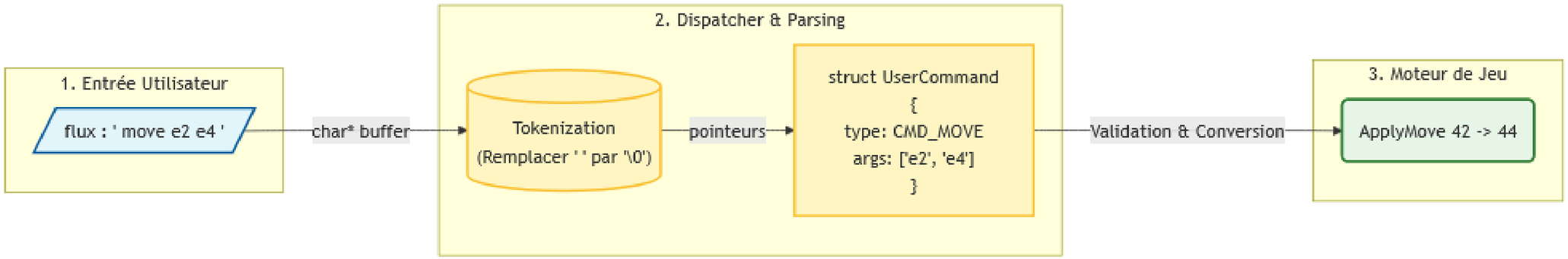
- GtkDragSource
- GtkDropTarget

CLI

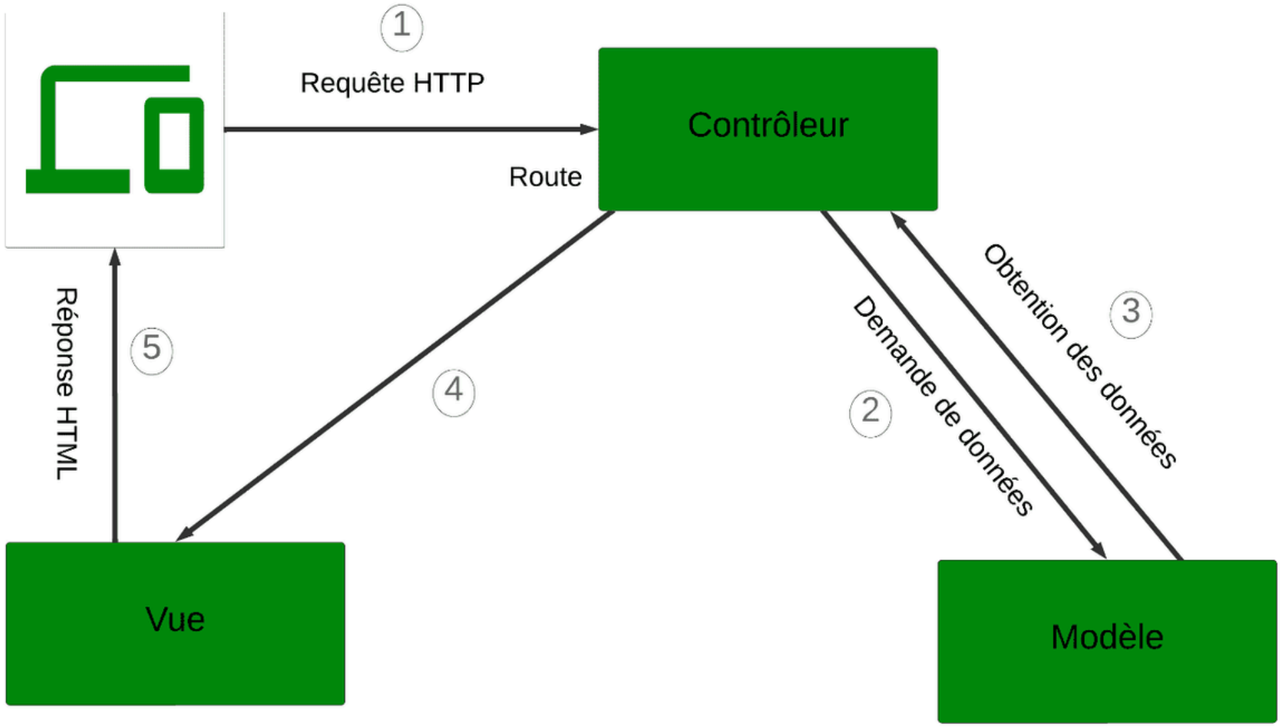

Application
-GameConfig config
-GameEngine* engine
-bool is_running
-char* current_filename
+Init(int argc, char** argv)
+Run()
+Cleanup()
+RequestQuit()
GameConfig
+int port
+bool verbose
+string mode
+ParseArgs(argc, argv)
+LoadFromRCFile(path)
Shell
-char* prompt
+InitReadline()
+ReadInput() : : UserCommand
-AttemptCompletion(text, start, end)
-AddHistory(line)
CommandDispatcher
+Execute(app, UserCommand cmd)
-HandleMove(app, args)
-HandleSave(app, args)
-HandleLoad(app, args)
-HandleHelp()
-HandleQuit(app)
Creates
Consumes
Controls
«struct»
UserCommand
+CommandType type
+char[3][64] args
+int arg_count
Serializer
+SaveGame(app, filename) : : bool
+LoadGame(app, filename) : : bool
-SerializeBoard(bitboards)
-ParseLine(line)
«Module Engine»
GameEngine
+Init()
+ApplyMove(Move m)
+GetLegalMoves()
+Undo()

CLI

Exemple d’Algorithme



Architecture



Conclusion et Ouverture

Références bibliographiques

image 1: https://cf.geekdo-images.com/lJwtJPMMJDHyh5VQEtQ0Qg__opengraph/img/28qvxq0JEzEKfKdUkJ-soUXD3w8=/fit-in/1200x630/filters:strip_icc()/pic1152391.jpg
image 2: https://cas.stthomas.edu/_media-library/_images/history/1920x1080/stthomas-history-time.jpg
image 3 : https://media.licdn.com/dms/image/v2/D4E12AQFz43ASmCZy2Q/article-cover_image-shrink_600_2000/article-cover_image-shrink_600_2000/0/1662135421412?e=2147483647&v=beta&t=3aaBzGQbZrcjTwjk57uClMbs_FlJpKuIIeBBx7uF90Q
image 4 : https://bodogemu.com/24/amazons-start-board.png
image 5 :https://bodogemu.com/24/amazons-running-game-256.png
image 6: https://static.alphorm.com/blog-alphorm-uploads/2025/01/03193647/schema-architecture-mvc-laravel-78.png
image 7 : https://preview.redd.it/hexagonal-vs-clean-vs-onion-architecture-which-is-truly-the-v0-75onmecznf0g1.png?width=481&format=png&auto=webp&s=dbcaae9872dfd77562e5e651c6a09def2e6b5a98
image 8 :https://preview.redd.it/hexagonal-vs-clean-vs-onion-architecture-which-is-truly-the-v0-b9sssngznf0g1.png?width=1050&format=png&auto=webp&s=9ec963b4b0bd9949c01006d2779eb326dfc5756b
Tree example image : https://www.researchgate.net/publication/380934806_Code_Repair_with_LLMs_gives_an_Exploration-Exploitation_Tradeoff

Références d’informations :
[1] Louis Victor Allis. Searching for Solutions in Games and Artificial Intelligence. Ph.D. Thesis. 1994. url : http://fragrieu.free.fr/SearchingForSolutions.pdf (visité le 21/01/2026).
[2] Patrick Hensgens. A Knowledge-based Approach of the Game of Amazons. Master’s Thesis. 2001. url : https://project.dke.maastrichtuniversity.nl/games/files/msc/Hensgens_thesis.pdf (visité le 20/01/2026).
[3] John Tromp et Gunnar Farnebäck. Combinatorics of Go. 2007. url : https://tromp.github.io/go/gostate.pdf (visité le 21/01/2026).
[4] GeeksforGeeks. Left Child Right Sibling Representation of Tree. 2024. url : https://www.geeksforgeeks.org/left- child- right- sibling-representation-tree/ (visité le 21/01/2026).